

Machine Learning Technologies

k-Nearest Neighbours

Metrics

Donny Hurley

ATU Galway

Lazy and Eager Learning

- Lazy: wait for query before generalizing
 - kNearest Neighbour, Case based reasoning
- Eager: generalize before seeing query
 - Radial basis function networks, ID3, Backpropagation, NaiveBayes, ...
- Does it matter?
 - Eager learner must create global approximation, but Lazy learner can create many local approximations
- if they use same H , lazy can represent more complex fns (e.g., consider $H =$ linear functions)



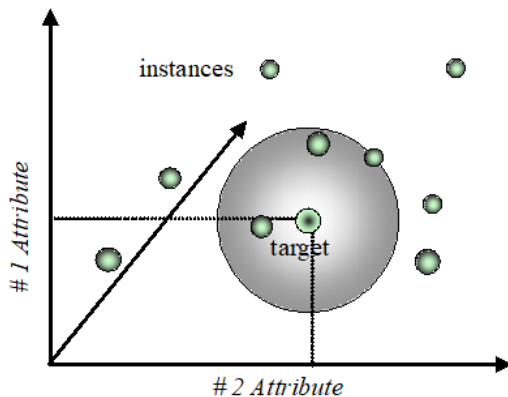
kNN is basically a two-part algorithm

- ① Retrieve the k most similar recorded cases
- ② “Average” across these k cases (regression)
 - take the max of k votes for ordinal types (classification)

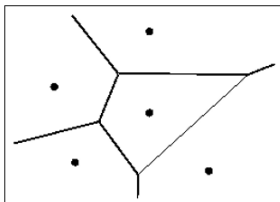
so kNN can be used for either regression or classification. However, let's ignore regression and stick to classification!

Distance

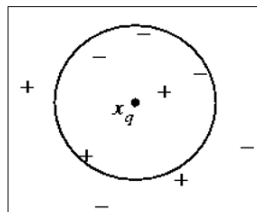
May require examining all instances



Voronoi Diagram



1NN Boundaries



KNN boundary

We typically need a notion of "distance" to calculate the "nearest" neighbour. Often use Euclidean distance i.e. (with p predictor values, $\mathbf{t}$ is a target and $\mathbf{s}$ is a source, t_1 is the first attribute for $\mathbf{t}$, t_2 etc.

$$d(\mathbf{t}, \mathbf{s}) = \sqrt{(t_1 - s_1)^2 + \dots + (t_p - s_p)^2}$$

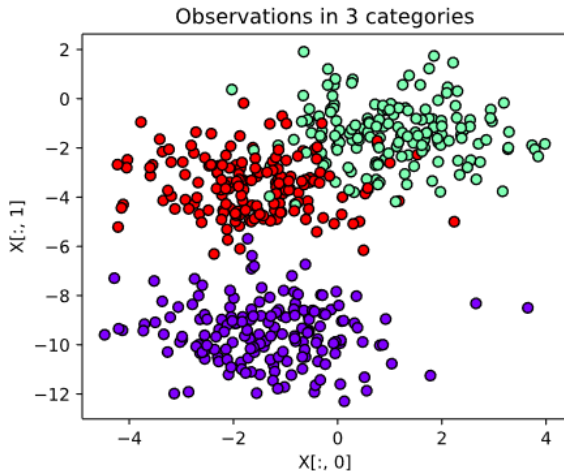
Caution: Scale matters. If one predictor has a much larger value than another, it will contribute more to the distance measurement.

Formalising everything

- 1 If you have a training set of data $(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m)$.
- 2 Given a target $\mathbf{x}_j$.
- 3 Calculate all $d(\mathbf{x}_j, \mathbf{x}_i)$ and select the k smallest ones.
- 4 Classify/Approximate $\mathbf{x}_j$ based on the k nearest $\mathbf{x}_i$ you found.

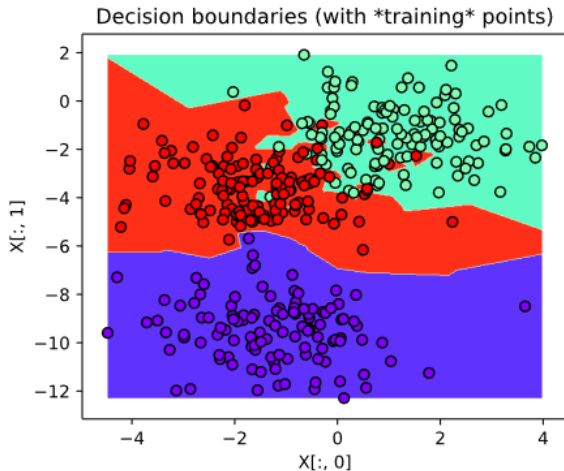
Nearest Neighbour

What if the prediction is made by just looking at which training point is it closest to?



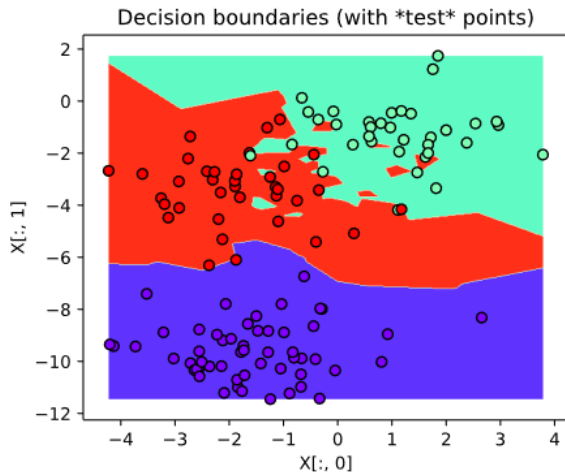
Nearest Neighbour

Not too bad. Looks like overfitting the training data.



Nearest Neighbour

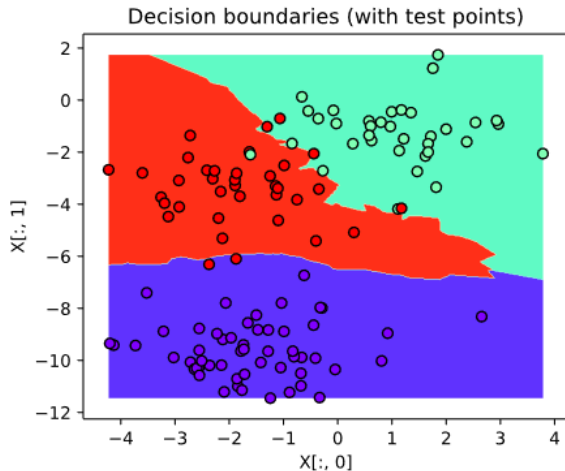
With the validation data, can see noise.



- Instead of looking at the one nearest training point, we could look at k and have a vote: a k -nearest neighbours classifier.
- e.g. with $k = 5$, we might want to predict a category for a point and find that the nearest five training points are green, green, red, green, red. Take the most common and predict green.

k Nearest Neighbour

Smooths out some of the overfitting



Of course we want to do this by programming. It follows the exact same procedure as the others. Model, fit, evaluate.

```
1 from sklearn.neighbors import KNeighborsClassifier
2 model = KNeighborsClassifier(n_neighbors=5)
3 model.fit(X_train, y_train)
4 print(model.score(X_val, y_val))
5
6 0.944
```

which is quite a good result for such a simple technique

There's tradeoff. If k is too small, you'll overfit the training data. If it's too large, you'll underfit reality.

Any guesses?

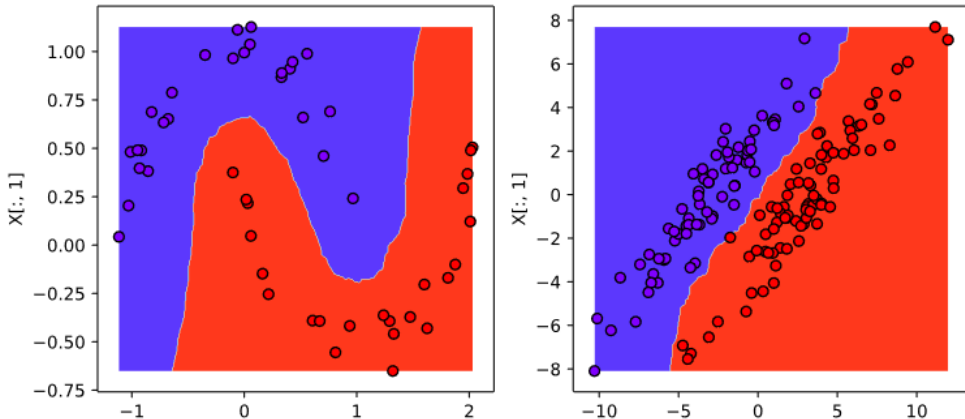
Cross-validation. Experiment.

So you can use k -fold cross validation to choose the k in k -nearest neighbours....is that confusing? They are two different k 's!

We are choosing the hyperparameter k for k -nearest neighbours and you apply say a 10-fold cross validation procedure to choose it

kNN can create a complicated boundary shape (both with `KNeighborsClassifier(9)`):

Decision boundaries (with test points)



- Imagine instances described by 20 attributes, but only 2 are relevant to target function
- Curse of dimensionality: nearest nbr is easily misled when highdimensional X

High-dimensionality increases sparsity. The more features, the further the distance apart are going to be

Remark on Nearest Neighbours

One further remark on nearest neighbours

Since the calculations are done when a new data point is attempting to be classified, the model must store all the training information. (Lazy)

Storing all the training information can be quite wasteful, better generalisation models would not have to store the full training set.

k -nearest neighbours is very quick to train (it does not do anything except store the set), however prediction is slow

It also will take up more memory than Eager models. Eager models just store the weights, Lazy ones store the entire dataset.

- It's easy to show examples that are 2D points: they make nice figures and are easy to visualize, but they aren't so realistic.
- Let's look at some more realistic data: there are lots of example data sets out there for ML experimentation: mldata.org, UCI Machine Learning Repository, KDnuggets dataset list.
- Scikit-learn has some data sets built-in. (like the diabetes and iris datasets that we've seen)

```
1 from sklearn.datasets import load_breast_cancer
2 bc = load_breast_cancer()
3 print(bc.data.shape)
4 print(bc.target.shape, np.unique(bc.target))
5 print(bc.feature_names)
6 X = bc.data
7 y = bc.target
```

```
1 (569, 30)
2 (569,) [0 1]
3 ['mean radius' 'mean texture' 'mean perimeter' 'mean area'
4  'mean smoothness' 'mean compactness' 'mean concavity'
5  'mean concave points' 'mean symmetry' 'mean fractal dimension'
6  'radius error' 'texture error' 'perimeter error' 'area error'
7  'smoothness error' 'compactness error' 'concavity error'
8  'concave points error' 'symmetry error' 'fractal dimension error'
9  'worst radius' 'worst texture' 'worst perimeter' 'worst area'
10 'worst smoothness' 'worst compactness' 'worst concavity'
11 'worst concave points' 'worst symmetry' 'worst fractal dimension']
```

Apply a randomly-chosen classifier and it does quite well

```
1 model = GaussianNB()
2 model.fit(X_train, y_train)
3 print(model.score(X_val, y_val))
4 print(classification_report(y_val, model.predict(X_val)))
5
6 0.944055944056
7
8      precision    recall  f1-score   support
9
10     0       0.94      0.91      0.93        55
11     1       0.94      0.97      0.96        88
12
13 avg / total       0.94      0.94      0.94       143
```

Although the 5.6% inaccuracy may be too much for this type of information. Ignore this and just think of modelling.

kNN seems to do worse

```
1 model = KNeighborsClassifier(n_neighbors=9)
2 model.fit(X_train, y_train)
3 print(model.score(X_val, y_val))
4
5 0.923076923077
```

Any ideas why?

Ask questions about the dataset, how it fits into the model and how the model reacts. Remember I said about distance metrics being heavily influenced by *larger* values, so we may need some form of scaling.

The scales are completely different. Measuring "nearest" counts "mean radius" as much more important than "worst concavity".

```
1 bc_df = pd.DataFrame(bc.data, columns=bc.feature_names)
2 print(bc_df[['mean radius', 'texture error', 'worst concavity']].
   describe())
3
4      mean radius  texture error  worst concavity
5 count      569.000000      569.000000      569.000000
6 mean        14.127292         1.216853         0.272188
7 std          3.524049         0.551648         0.208624
8 min          6.981000         0.360200         0.000000
9 25%         11.700000         0.833900         0.114500
10 50%         13.370000         1.108000         0.226700
11 75%         15.780000         1.474000         0.382900
12 max         28.110000         4.885000         1.252000
```

At the very least, we can bring the features into a similar range.

This is a common transformation step at the start of a model: scale each feature separately so they have the same "size". That could mean scaling your data so:

- the smallest value is 0 and the largest is 1.
- the mean is 0 and standard deviation is 1.
- something else that makes sense for your data.

sklearn of course has a transformer

```
1 from sklearn.preprocessing import MinMaxScaler, StandardScaler
2 model = make_pipeline(
3     StandardScaler(),
4     KNeighborsClassifier(n_neighbors=9)
5 )
6 model.fit(X_train, y_train)
7 print(model.score(X_val, y_val))
8
9 0.958041958042
```

An improvement on both GaussianNB and the previous KNeighborsClassifier

Evaluating Machine Learning Models

How predictive is the model we learned?

- For regression, usually
 - Mean Squared Error (MSE) : What we used for linear regression
 - R^2
 - p-values (Hypothesis testing)
 - F-stats (ANOVA)
 - AIC/BIC
- For classification, many options
 - ROC curve
 - Lift Chart
 - Accuracy score
 - F1 Score, Recall, Precision

Choosing a metric

- The procedure of systematically choosing a set of predictors that have a significant relationship with the response variable is called variable selection.
- But which metric (F-stats, p-values [hypothesis testing], R^2 , AIC/BIC) should we use to determine the significance of a set of predictors?
- Rather than relying on a single metric, we should use multiple metrics in conjunction and double check with common sense!

For evaluating Classification algorithms often people just see how accurate the predictions are on the test set

```
1 y_pred = model.predict(X_val)
2 from sklearn.metrics import accuracy_score
3 accuracy_score(y_val, y_pred)
```

Will give a score (≤ 1) which is the fraction of how many predictions were correct.

This stores the y predictions separately (which may be useful later) but the model has a method itself that can be used if you just want the accuracy score

```
1 model.score(X_val, y_val)
```

This one line does the predicting and comparing of predictions in the same step.

Accuracy score is a sometimes called a very *harsh* metric and it does not tell us how a classifier performs for particular classes.

The number digit classifier has 10 possible classes, i.e. 0,1,2,3,4,5,6,7,8,9. Maybe we just care about the performance of the number 7.

Also, is it over-predicting some of the classes? (Is it falsely identifying 1's as 7, i.e. is it a false-positive for 7 and a false-negative for 1)

Precision/Recall

For each category we can measure two important metrics, *Precision* and *Recall*

Precision: how many selected were correct.

Recall: how many correct were found? A Classification Report can be outputted from our test data

```
1 from sklearn.metrics import classification_report
2 print(classification_report(y_val, y_pred))
3
4           precision    recall  f1-score   support
5
6      0           1.00       1.00       1.00         54
7      1           0.89       0.94       0.92         35
8      2           0.94       0.89       0.91         36
9
10 avg / total           0.95       0.95       0.95        125
```

		precision	recall	f1-score	support
2	0	1.00	1.00	1.00	54
3	1	0.89	0.94	0.92	35
4	2	0.94	0.89	0.91	36

The above is showing the performance for 3 categories (0, 1, 2). The support is the number of occurrences of each class in `y_true`.

Category 1 has precision 0.89. This means that 89% of items predicted to be in Category 1 were correct. The other 11% were incorrectly identified as Category 1 (false-positive for category 1)

Category 1 has recall 0.94. This means that 94% of the items that should've been predicted in Category 1 were found. The other 6% were incorrectly identified in some other category (false-negative for category 1)

- **Precision** is defined as the number of relevant domains retrieved, divided by the total number of domains (in that category) retrieved.

$$\text{Precision} = \frac{\# \text{relevant-retrieved}}{\# \text{total-documents}}$$

- **Recall** is defined as the number of relevant domains retrieved divided by the total number of relevant domains (in that category).

$$\text{Recall} = \frac{\# \text{relevant-retrieved}}{\# \text{total-relevant}}$$

- sklearn says

$$\text{precision} = \frac{tp}{tp + fp} \quad \text{recall} = \frac{tp}{tp + fn}$$

where tp is number of true positives, fp is number of false positives and fn is number of false negatives

Cancer Example

Example

Suppose we build a model for cancer diagnosis and we have a validation sample of 100 patients. We have the following table describing the performance of our model (this is called a confusion matrix, I will come back to this):

		Predicted	
		Cancer = Yes	Cancer = No
Actual	Cancer = Yes	True Positive (TP) = 25	False Negative (FN) = 5
	Cancer = No	False Positive (FP) = 5	True Negative (TN) = 65

We can read precision from the columns.. Precision for Cancer Yes = $\frac{25}{25+5} = 0.83$.

Precision for Cancer No = $\frac{65}{65+5} = 0.928$.

We can read recall from the rows... Recall for Cancer Yes = $\frac{25}{25+5} = 0.83$. Recall for Cancer No = $\frac{65}{65+5} = 0.928$.

Note: Accuracy works quite well when the class distribution is similar but very often our data is imbalanced (cancer analysis, we hopefully have many more patients without cancer than with - imbalanced set).

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

Accuracy is used when the True Positives and True negatives are more important

Example

The accuracy in this case is = 90% which is a high enough number for the model to be considered as 'accurate'. However, there are 5 patients who actually have cancer and the model predicted that they don't have it. Obviously, this is too high a cost. Our model should try to minimize these False Negatives.

F1-score is a better metric when there are imbalanced classes.

F1-score: The harmonic mean of Precision and Recall.

F1-Score gives a better measure of the incorrectly classified cases than the Accuracy.

IF we let P and R stand for precision and recall respectively then

$$F1 = 2 \frac{P \cdot R}{P + R}$$

		precision	recall	f1-score	support
1					
2	0	1.00	1.00	1.00	54
3	1	0.89	0.94	0.92	35
4	2	0.94	0.89	0.91	36

Extreme Example

Of all women who receive regular mammograms, about 10 percent will get called back for further testing and of those, only about 0.5 percent will be found to have cancer. So for the women that are called back 99.5% of them will not have cancer. So if my model is

```
1 def has_cancer(X):  
2     return false
```

Then my model will be 99.5% accurate for these women! But it misses all the ones that have cancer.

Confusion Matrix

Another way of evaluating the the accuracy of a classification is to compute a confuse matrix.

By definition a confusion matrix C is such that $C_{i,j}$ is equal to the number of observations known to be in group i but predicted to be in group j .

The diagonals are therefore the "correct" predictions, while the off diagonals is where something has been miscategorised

2-class problems are a particular case

	event	no-event
event	true positive	false positive
no-event	false negative	true negative

As an example in sklearn

```
1 from sklearn.metrics import confusion_matrix
2 >>> y_true = [2, 0, 2, 2, 0, 1]
3 >>> y_pred = [0, 0, 2, 2, 0, 2]
4 >>> confusion_matrix(y_true, y_pred)
5 array([[2, 0, 0],
6        [0, 0, 1],
7        [1, 0, 2]])
```

We can use confusion matrices to see where exactly our misclassifications are happening and if necessary tune our ML algorithm. Precision and Recall are read from a confusion matrix.

Colour Data Set

	black	blue	brown	green	grey	orange	pink	purple	red	white	yellow
black	68	0	1	0	2	0	0	1	1	0	0
blue	3	84	0	4	7	0	0	8	0	0	0
brown	5	0	18	4	5	1	1	0	18	0	2
green	4	4	3	135	8	0	0	0	0	0	1
grey	4	4	0	7	62	1	1	2	3	0	0
orange	0	0	1	2	0	16	2	0	7	0	2
pink	0	0	0	0	0	0	41	15	6	0	0
purple	5	5	0	1	3	0	2	103	5	0	0
red	2	6	1	6	5	3	10	3	26	0	2
white	2	0	1	0	4	0	2	0	0	0	0
yellow	0	1	1	4	4	0	2	0	0	0	17

We can read the precision for black off by the black column (predicted were correct).

68 predicted were correct, while $68 + 3 + 5 + 4 + 4 + 0 + 0 + 5 + 2 + 2 + 0 = 93$ were predicted in total to be black. So precision was $\frac{68}{93} = 0.7311$.

We can read the recall for purple by the purple row (correct were found).

103 were found, while $5 + 5 + 0 + 1 + 3 + 0 + 2 + 103 + 5 + 0 + 0 = 124$ should have been found in total. So recall for purple was $\frac{103}{124} = 0.83$.

Optimising to these scores?

Written a lot about how we want to get the best metrics: f1-score, accuracy etc.
However, we can't actually do this directly!

- Metrics on a dataset is what we care about (performance)
- We typically cannot directly optimize for the metrics
- Our loss/cost function should reflect the problem we are solving. We then hope it will yield models that will do well on our dataset

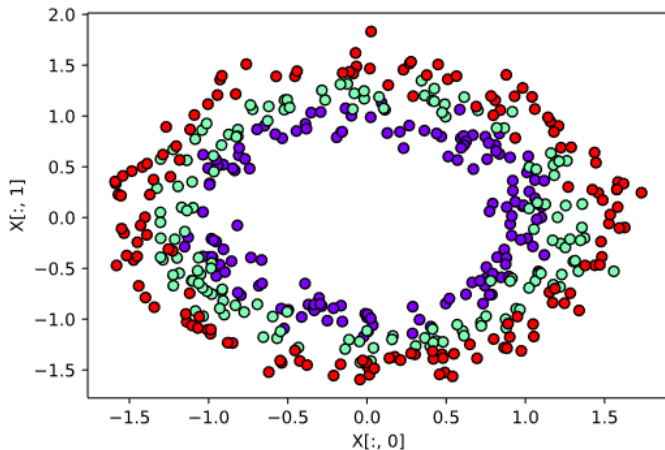
- Not surprisingly, the kind of X values we have for each training point have a huge effect on what we can learn/predict.
- The components of each X value are the *features*.
- A combination of understanding our data and the model can help us be much more successful.

We have seen:

- Just use the original features.
- Calculate powers of the original features (`PolynomialFeatures`).
- Scale features to tidy their min/max (`MinMaxScaler`) or mean/stddev (`StandardScaler`).
- Calculate anything else we want/need (`FunctionTransformer`).

Feature Engineering

We can design whatever features we think will give the model something to work with.
As an example:

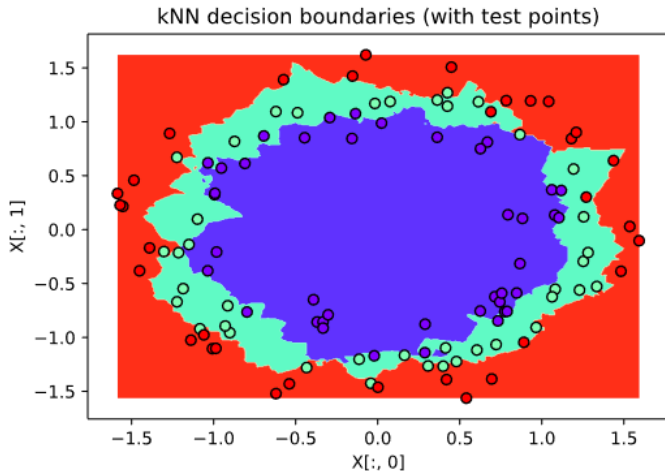


With that picture and a kNN model:

```
1 model = KNeighborsClassifier(7)
2 model.fit(X_train, y_train)
3 print(model.score(X_val, y_val))
4
5 0.805309734513
```

Feature Engineering

It doesn't have enough training data to really capture what's going on, and overfits the training data.



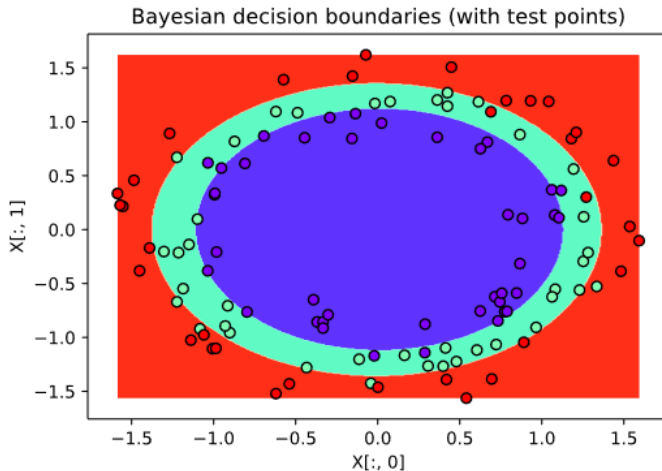
Maybe if it has a *feature* of how far away the point is from (0,0) it would improve the results.

Add a transformer:

```
1 def add_radius(X):
2     X0 = X[:, 0]
3     X1 = X[:, 1]
4     R = np.linalg.norm(X, axis=1) # Euclidean norm
5     return np.stack((X0, X1, R), axis=1)
6
7 model = make_pipeline(
8     FunctionTransformer(add_radius),
9     GaussianNB())
10 model.fit(X_train, y_train)
11 print(model.score(X_val, y_val))
12
13 0.893805309735
```

Feature Engineering

The model does better with the right features



Other problems that may arise is having

- More features than train+test data
- Too large of a data set (processor/memory)

Sometimes we have to group common features together

Example

We observe multiple weather values (rainfall, temperature) every day. This gives us 365 points (times the number of weather features) for one year. It may make sense to average these values for each month and cut down the number of points to 12 for a year (times the number of weather features). This comes from knowing the data. We could also average over each day in a month, 1st of month, 2nd of month etc. so down to 31 points....however this would make no sense for grouping together.

High-Dimensional Data kNN (Source Wikipedia)

- For high-dimensional data (e.g., with number of dimensions more than 10) dimension reduction is usually performed prior to applying the k-NN algorithm in order to avoid the effects of the curse of dimensionality.
- The curse of dimensionality in the k-NN context basically means that Euclidean distance is unhelpful in high dimensions because all vectors are almost equidistant to the search query vector.
- Feature extraction and dimension reduction can be combined in one step using principal component analysis (PCA), linear discriminant analysis (LDA), or canonical correlation analysis (CCA) techniques as a pre-processing step, followed by clustering by k-NN on feature vectors in reduced-dimension space. This process is also called low-dimensional embedding.
- For very-high-dimensional datasets (e.g. when performing a similarity search on live video streams, DNA data or high-dimensional time series) running a fast approximate k-NN search using locality sensitive hashing, "random projections", "sketches" or other high-dimensional similarity search techniques from the VLDB toolbox might be the only feasible option.